Análisis de Utilidad de Variables (Iowa Liquor Sales)

Enfoque: Modelado de Entornos Gymnasium para Deep Reinforcement Learning (DRL)

Al construir un Gemelo Digital logístico, las variables del dataset transaccional se mapean directamente a los componentes matemáticos de un Proceso de Decisión de Markov (MDP). A continuación, se detalla la utilidad estratégica de cada columna de la base de datos para la tesis.

1. El Motor del Tiempo y el Estado (S_t)

Estas variables definen el paso del tiempo en la simulación y la identificación del producto a controlar.

- `ordered_on` **(Date):** * **Utilidad RL:** Define el *Time Step* (t) del entorno. El agente tomará una decisión en cada paso t (ej. diario o semanal). Agrupar los datos por esta columna nos da la serie de tiempo.
- `item_no` y `im_desc` **(String):**
  - **Utilidad RL:** Filtro de contexto. Inicialmente, entrenaremos un agente de un solo producto (Single-Item Inventory). Usaremos estas columnas para aislar el producto con mayor rotación (ej. el vodka más vendido) y crear un entorno estable.
- `category_name` **(String):**
  - **Utilidad RL:** Permite agrupar productos sustitutos. Si un producto se agota, ¿el cliente compra otro de la misma categoría? Útil para modelos avanzados de demanda correlacionada.

2. El Entorno Estocástico (Demanda)

Esta es la fuente de incertidumbre. El entorno cambia de estado basado en lo que el cliente hace, no solo en lo que el agente decide.

- `sales_bottles` **(Float -> Int):**
  - **Utilidad RL:** Es la **Demanda Real (D_t)**. Es la variable más crítica. El agente no la conoce antes de tomar su decisión de compra. Si el Inventario del agente es menor que `sales_bottles`, ocurre un *Stockout* (Rotura de stock).

3. Función de Recompensa (R_t)

El agente de DRL aprende maximizando recompensas numéricas. Estas variables construyen nuestra función objetivo.

- `sales_dollars` **(Float):**
  - **Utilidad RL:** Ingreso Bruto. Si dividimos `sales_dollars / sales_bottles`, obtenemos el **Precio de Venta Unitario (p)**.
  - **Ecuación de Recompensa:**
    $R_t = (Unidades_Vendidas \times p) - Costos_de_Inventario.$

- *Nota sobre Costos:* Como el dataset actual (vista 1051) no incluye el costo de compra original de la botella (`state_bottle_cost` en versiones anteriores), usaremos `sales_dollars` para estimar un margen de ganancia estándar (ej. 30%) y castigar al agente basándonos en el valor del inventario inmovilizado.

4. El Espacio de Acción (A_t) y Restricciones Físicas

Definen las reglas del juego. ¿Qué puede hacer el agente y cuáles son los límites físicos del almacén?

- `pack` **(Integer):**
  - **Utilidad RL:** Define el **Espacio de Acción Discreto**. En la vida real, los minoristas no piden "3 botellas" a la destilería, piden "Cajas". Si `pack` es 12, y la acción del agente es $A_t = 2$, el inventario real aumentará en $2 \times 12 = 24$ botellas.
- `bottle_volume_ml` **(Float) /** `sales_liters` **/** `sales_gallons` **:**
  - **Utilidad RL:** Límite de Capacidad (C_{max}). Los almacenes no tienen espacio infinito. Podemos usar el volumen en litros para penalizar fuertemente al agente (recompensa negativa grande) si intenta pedir más producto del que físicamente cabe en los metros cúbicos de la bodega.

5. Complejidad de Red (Para Modelos Multi-Agente / Multi-Echelon)

Variables que escalan la dificultad del simulador para fases futuras de la tesis.

- `vendor_name` **y** `vendor_number` **(String):**
  - **Utilidad RL:** Modelado de Proveedores y *Lead Times* (Tiempos de entrega). Podemos simular que diferentes proveedores tardan diferentes cantidades de días en entregar.
- `store_no` **,** `store_name` **,** `store_city` **,** `county_name` **:**
  - **Utilidad RL:** Topología de la Cadena de Suministro.
    - *Nivel 1 (Fácil):* Sumar todas las tiendas para simular un único Almacén Central.
    - *Nivel 2 (Experto):* Cada `store_no` es un agente independiente que compite por el inventario del proveedor central, simulando una red logística completa (Multi-Agent RL).